EDtoolbox – input data structs

`EDres = EDmain_convex(geoinputdata,Sinputdata,Rinputdata,envdata,controlparameters,filehandlingparameters)`

EDmain_convex - program flow

Each stage saves its result in a file `<filestem>_<suffix>.mat`, which is reused when the input data for that stage have not changed.

```
EDcheckinputstructs          → checked input structs, _settings
EDedgeo                      → planedata, edgedata (_eddata)
EDgensurfreceivers           → surface receivers (tf_ESIEBEM)
EDSorRgeo                    → Sdata (_Sdata)
EDSorRgeo                    → Rdata (_Rdata)
EDfindconvexGApaths          → firstorderpathdata (_paths)
EDmakefirstordertfs          → tfdirect, tfgeom, tfdiff (_tf)
EDmakefirstorderirs          → irdirect, irgeom, irdiff (_ir)
EDed2geo                     → edgetoedgedata (_ed2data)
EDinteg_submatrixstructure   → Hsubmatrixdata (_submatrixdata)
EDintegralequation_convex_tf → tfinteqdiff (_tfinteq, _qfinal)
EDpropagateESIEBEM           → tftot_ESIEBEM (_tfESIEBEM)
EDfindHODpaths               → hodpaths (_hodpaths)
EDmakeHODirs                 → irhod (_irhod)
```

All results are collected in `EDres` (also saved as `_EDres`). In Matlab: `EDinputdoc` describes the input fields, `EDdatastructs` the internal data structs, and `EDgetresults` loads saved results.

geoinputdata

Exactly one geometry source must be given (unless `.freefieldcase` = 1): `.geoinputfile`, or `.corners` and `.planecorners`.

- `.freefieldcase` [0] 1: no scattering object; only the direct sound is computed.
- `.geoinputfile` [**required**] The name of a geometry file (`.cad`). Either this, or `.corners` and `.planecorners`, must be given.
- `.corners` [**required**] [ncorners,3] the corner coordinates (instead of a geometry file). If a geometry file is given, it is filled in from the file.
- `.planecorners` [**required**] [nplanes,nmaxcorners] the corner numbers of each plane; the order of the corners defines the front side of the plane (instead of a geometry file). If a geometry file is given, it is filled in from the file.
- `.planerefltypes` [*derived*] [nplanes,1] for each plane: +1 rigid, 0 totally absorbing, -1 pressure release. Only with `.corners`/`.planecorners`. Default: all rigid.
- `.firstcornertoskip` [1e6] Edges with at least one corner with this number, or higher, are switched off.
- `.listofcornerstoskip` [[]] Edges with at least one corner in this list are switched off.
- `.listofedgestoskip` [[]] Edge numbers (one column), or corner pairs (two columns), of edges that are switched off.
- `.planeseesplanestrategy` [0] 1: the plane-to-plane visibility is checked only between the plane midpoints (enough for some geometries). 0: a full check.
- `.allownonconvex` [0] 1: a non-convex model is accepted (`EDmain_convex` computes correct results only for convex models).

Sinputdata

- `.sourcetype` ['monopole'] `'monopole'` or `'polygonpiston'` (see the piston fields below).
- `.coordinates` [**required**] [nsources,3] the source coordinates. Required for monopoles; for pistons, the center points are computed.
- `.doaddsources` [0] 1: the contributions of all the sources are added. 0: each source gives a separate result. For a single source, it is always 1.
- `.doallSRcombinations` [1] 1: all source-receiver combinations are computed. 0: only source N to receiver N (then there must be as many sources as receivers).
- `.nedgesubs` [2] The number of points per edge that are used in the visibility tests (2 = the two end points).
- `.sourceamplitudes` [1] A scalar, a row ([1,nfrequencies]), a column ([nsources,1]) or the full matrix [nsources,nfrequencies]; it is expanded to the full size. For `'ir'`, one value per source.
- `.pistoncornercoordinates` [**required**] [npistoncorners,3] the corner coordinates of all the pistons. Required for polygon pistons.
- `.pistoncornernumbers` [**required**] [npistons,nmaxcorners] the corner numbers of each piston (0 for unused positions). Required for polygon pistons.
- `.pistonplanes` [**required**] [npistons,1] the number of the plane that each piston is on. Required for polygon pistons.
- `.pistongaussorder` [3] The quadrature order on each piston: a triangle rule of degree 0-8 for triangles, order*order Gauss-Legendre points for quadrilaterals, and the triangle rule on each triangle from the center point for polygons with more corners (which must be convex).

For polygon pistons, the fields `.coordinates`, `.pistongausscoordinates`, `.pistongaussweights`, `.ncornersperpiston` and `.pistonareas` are computed (see help `EDdatastructs`).

Rinputdata

- `.coordinates` [**required**] [nreceivers,3] the receiver coordinates. Required.
- `.nedgesubs` [2] The number of points per edge that are used in the visibility tests (2 = the two end points).

envdata

- `.cair` [344] The speed of sound, in m/s.
- `.rhoair` [1.21] The density of air, in kg/m3.

controlparameters

- `.generalmethod` [**required**] The calculation method: `'tf'` = transfer functions (ESIE), `'ir'` = impulse responses, `'tf_ESIEBEM'` = transfer functions via surface receivers (ESIEBEM).
- `.frequencies` [**required**] The frequencies, in Hz. Required for `'tf'` and `'tf_ESIEBEM'`.
- `.fs` [44100] The sampling frequency, in Hz. Only for `'ir'`.
- `.Rstart` [0] A distance, in m, that gives the time zero (IRs) or the phase reference (TFs): a path of this length has no delay.
- `.directsound` [1] 1: the direct sound is included. 0: it is not.
- `.skipfirstorder` [0] 1: the direct sound, the specular reflections and the first-order diffraction are not computed.
- `.difforder` [**required**] The highest diffraction order. Required. With the GMRES methods, any difforder > 2 gives the full solution, with all diffraction orders included.
- `.diff1method` ['accurate'] The method for first-order diffraction: `'accurate'` (adaptive integration, with an analytic treatment near the apex point) or `'GL'` (Gauss-Legendre quadrature, faster).
- `.diff1GLorder` [16] The number of Gauss-Legendre points per edge, for diff1method = `'GL'`.
- `.inteqsolutionmethod` ['GMRES_fast'] How the integral equation is solved: `'GMRES_fast'` (with an explicit sparse matrix), `'GMRES_large'` (matrix-free, for large problems) or `'iterative_fixeddifforder'` (difforder-2 terms of the Neumann series). For difforder = 2, the iterative method is always used.
- `.inteqtargettol` [1e-10] The relative tolerance for GMRES.
- `.inteqgmresmaxiter` [100] The maximum number of GMRES iterations.
- `.ESIEsourcetermimplementation` ['accurate'] The source term of the integral equation: `'accurate'` uses singularity subtraction when a source is close to the plane of a face; `'legacy'` is the earlier method.
- `.ESIEpropagationimplementation` ['exactpole'] The propagation from the edge sources to the receivers: `'exactpole'` integrates the near-singular terms close to the zone boundaries with their exact complex poles (product integration), which converges regularly, as 1/ngauss^2; `'accurate'` uses singularity subtraction with a Taylor model of the poles, which leaves an irregular error for receivers within a few degrees of the plane of a face; `'legacy'` uses plain Gauss-Legendre quadrature. `'exactpole'` is not implemented for thin holes (`'accurate'` is used).
- `.ESIEpropagationrefine` [1] Advanced: the solution of the integral equation is refined to more points per edge (Nystrom interpolation, `EDrefineQvec`) before the propagation to the receivers, which is then more accurate. A factor for all edges (1 = no refinement), or rows [maxlength factor]: the edges not longer than maxlength get the factor (the smallest maxlength that fits); longer edges are not refined. The new number of points is ceil(factor*npoints). The solution and the `_qfinal` file are not changed.
- `.skipreceiverpropagation` [0] 1: the integral equation is solved, but the propagation to the receivers is skipped (the result is zero).
- `.ngauss` [16] The number of Gauss-Legendre points on the longest edge; shorter edges get proportionally fewer (at least 2). A negative value gives an odd number on every edge. The log file reports the points per wavelength.
- `.ngaussmin` [2] The minimum number of Gauss-Legendre points on every edge, e.g. for very short edges near the corners (odd if ngauss < 0).
- `.nedgeelemsuser` [[]] Advanced: the number of Gauss-Legendre points on chosen edges, overriding ngauss and ngaussmin. Rows [maxlength npoints]: the edges not longer than maxlength get npoints (the smallest maxlength that fits). Or rows [corner1 corner2 npoints]: the edge between these corners (rows of geoinputdata.corners).
- `.discretizationtype` [2] The edge discretization: 2 = Gauss-Legendre (the only one that is supported).
- `.surfacegaussorder` [5] The quadrature order on the largest surface, for the surface receivers. Only for `'tf_ESIEBEM'`.
- `.HODelemsize` [*derived*] For each diffraction order N, the edge element length is (cair/fs)/HODelemsize(N). Only for `'ir'`. Default: 2*2.^(-[0:difforder-1]).
- `.savealldifforders` [0] 1: the higher-order diffraction IRs are kept separate for each order. Only for `'ir'`.
- `.saveindividualfirstdiff` [0] 1: the first-order diffraction IR of each edge is kept separate. Only for `'ir'`.

filehandlingparameters

filehandlingparameters can be left out if a geometry file is given.

- `.outputdirectory` [**required**] The directory for all the result files. Required, unless a geometry file is given: then the default is the folder `'results'` next to it.
- `.filestem` [**required**] The first part of the names of all the result files. Required, unless a geometry file is given: then the default is its name.
- `.suppressresultrecycling` [0] 1: saved stage files are never reused, so everything is computed.
- `.showtext` [1] 0: no text on the screen. 1: one line per stage. 2 or more: more details.
- `.parallelworkers` [0] The largest number of parallel workers (Parallel Computing Toolbox) for the propagation to the receivers in the integral equation stage. 0: no parallel computing. Inf: all the workers of the parallel pool, which is started if needed. The results are the same; it only affects the computing time.
- `.saveSRdatafiles` [1] 1: the stage files `_Sdata` and `_Rdata` are saved.
- `.saveeddatafile` [1] 1: the stage file `_eddata` is saved.
- `.saveed2datafile` [1] 1: the stage file `_ed2data` is saved.

`.savesubmatrixdata` [1] 1: the stage file `_submatrixdata` is saved.

`.saveinteqsolution` [1] 1: the solution of the integral equation is saved in the stage file `_qfinal` (if it is smaller than 4 GB), so that it can be reused for other receivers.

`.savepathsfile` [1] 1: the stage file `_paths` is saved.

`.savediff2result` [0] 1: the integral equation result is also computed without the second-order diffraction (extraoutputdata.tfinteqdiff_nodiff2 in the `_tfinteq` file).

`.savelogfile` [1] 1: a log file is written.

`.savefullEDresfile` [1] 1: the file `_EDres` contains the result matrices. 0: they are left out (`EDgetresults` loads them from the stage files).

`.saveISEStree` [0] Not used by `EDmain_convex`.

`.savehodpaths` [0] 1: the stage file `_hodpaths` is saved. Only for `'ir'`.

Tips

Plane wave. To simulate an incoming plane wave with amplitude 1 and phase zero at the origin, place a monopole far away, at the distance d, and set `Sinputdata.sourceamplitudes` $= d$ and `controlparameters.Rstart` $= d$.

Recycling. Changing only the receivers reuses the integral equation solution (`_qfinal`), if `filehandlingparameters.saveinteqsolution` $= 1$.

Refining q. `controlparameters.ESIEpropagationrefine` refines the integral equation solution before it is propagated to the receivers. For studies of one solution at several refinements, apply `EDrefineQvec` to a saved `_qfinal` (advanced use).